

UNIVERSIDAD DE GRANADA

PERIFÉRICOS Y DISPOSITIVOS DE INTERFAZ HUMANA

CURSO 2025/2026

Práctica 6. Reconocimiento de imágenes

Autora:
Inés Ruiz Sánchez

20 de mayo de 2026



**UNIVERSIDAD
DE GRANADA**

Índice

1. Introducción y objetivos	2
2. Cuestiones a resolver	2
3. Configuración del entorno de trabajo	3
3.1. Ejemplo de caras	4
4. Ejercicios propuestos	6
4.1. Ejercicio 1	6
5. Anexo	8
5.1. Fichero <i>ejemplo-fotos-1.py</i>	8
5.2. Fichero <i>ejercicio1.py</i>	9
5.3. Fichero <i>ejemplo2.py</i>	10
5.4. Fichero <i>ejemplo3.py</i>	11

Índice de figuras

1. Proceso de instalación de Python en Windows.	3
2. Instalación de OpenCV.	3
3. Repositorio OpenCV	4
4. Imagen inicial	4
5. Ejecución de ejemplo	5
6. Ejecución de ejemplo con <code>scaleFactor = 1.05</code>	6
7. Imagen 1	7
8. Imagen 2	8
9. Resultado de <i>imagen1</i> con <code>scaleFactor=1.35, minNeighbors=6</code>	9
10. Resultado de <i>imagen2</i> con <code>scaleFactor=1.5, minNeighbors=6</code>	10
11. Resultado de <i>imagen2</i> con <code>scaleFactor=1.35, minNeighbors=6</code>	11

Listings

1. Instalación de OpenCV.	3
2. Librerías para el Ejercicio 1	6
3. Comprobación de parametro	6
4. Guardar imagen	7
5. Código completo del ejemplo1	8
6. Código completo del ejercicio1	9
7. Código completo del ejemplo2	10
8. Código completo del ejemplo3	11

1. Introducción y objetivos

Los **objetivos** concretos de esta práctica son:

- Conocer cómo funcionan las técnicas de reconocimiento de imágenes.
- Hacer un programa sencillo para realizar detección facial.

Repositorio en GitHub: <https://github.com/ruiz314/PDIH/tree/main/P6>.

2. Cuestiones a resolver

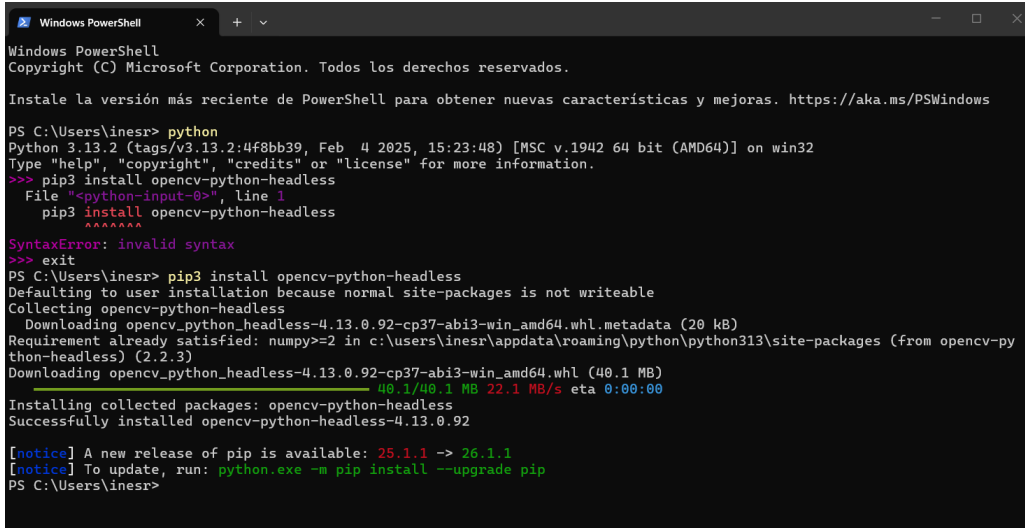
Para superar esta práctica se proponen los siguientes ejercicios:

1. Hacer reconocimiento de caras de personas en imágenes (ejemplo básico).
2. Hacer reconocimiento de caras de personas en vídeos.
3. Hacer reconocimiento de caras de gatos en vídeos.
4. Hacer simultáneamente el reconocimiento de cuerpos y caras de personas en vídeos. Es requisito que ambas cosas se detenten al mismo tiempo. Como sugerencia, se puede hacer una primera aproximación para hacer la detección en imágenes y posteriormente en vídeos. Para este ejercicio se facilita un vídeo de ejemplo, aunque se pueden usar otros diferentes.

3. Configuración del entorno de trabajo

El sistema operativo usado es Microsoft Windows. A continuación, se detallan los pasos seguidos para la instalación del software, la gestión de entornos virtuales y la adquisición de los modelos de detección necesarios.

Tenía instalada la **versión 3.13.2 de Python**, como se puede ver en la Figura 1.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\inesr> python
Python 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> pip3 install opencv-python-headless
File "<python-input-0>", line 1
    pip3 install opencv-python-headless
        ^^^^^^^
SyntaxError: invalid syntax
>>> exit
PS C:\Users\inesr> pip3 install opencv-python-headless
Defaulting to user installation because normal site-packages is not writeable
Collecting opencv-python-headless
  Downloading opencv_python_headless-4.13.0.92-cp37-abi3-win_amd64.whl.metadata (20 kB)
Requirement already satisfied: numpy>=2 in c:\users\inesr\appdata\roaming\python\python313\site-packages (from opencv-python-headless) (2.2.3)
Downloading opencv_python_headless-4.13.0.92-cp37-abi3-win_amd64.whl (40.1 MB)
  40.1/40.1 MB 22.1 MB/s eta 0:00:00
Installing collected packages: opencv-python-headless
Successfully installed opencv-python-headless-4.13.0.92

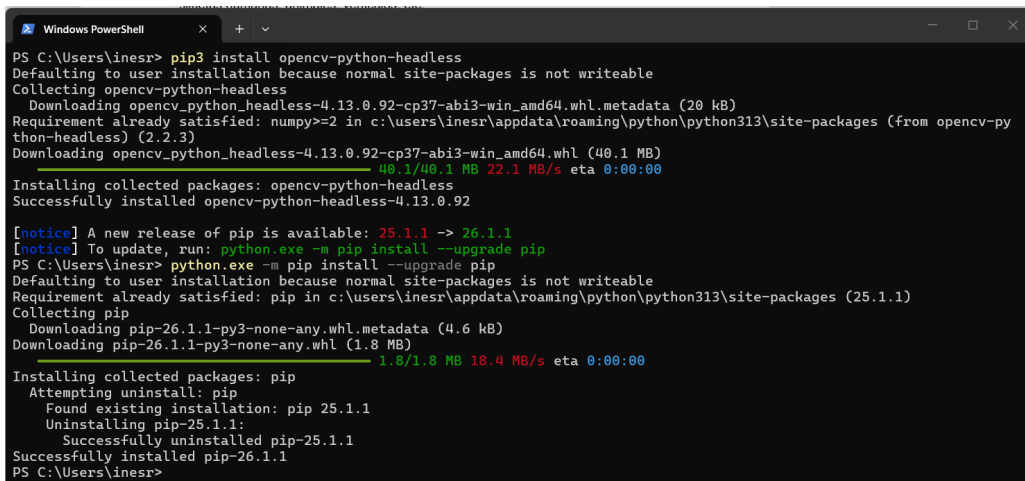
[notice] A new release of pip is available: 25.1.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\inesr>
```

Figura 1: Proceso de instalación de Python en Windows.

Para la instalación de la Librería **OpenCV-Python** uso la siguiente orden:

```
pip3 install opencv-python-headless
```

Listing 1: Instalación de OpenCV.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\inesr> pip3 install opencv-python-headless
Defaulting to user installation because normal site-packages is not writeable
Collecting opencv-python-headless
  Downloading opencv_python_headless-4.13.0.92-cp37-abi3-win_amd64.whl.metadata (20 kB)
Requirement already satisfied: numpy>=2 in c:\users\inesr\appdata\roaming\python\python313\site-packages (from opencv-python-headless) (2.2.3)
Downloading opencv_python_headless-4.13.0.92-cp37-abi3-win_amd64.whl (40.1 MB)
  40.1/40.1 MB 22.1 MB/s eta 0:00:00
Installing collected packages: opencv-python-headless
Successfully installed opencv-python-headless-4.13.0.92

[notice] A new release of pip is available: 25.1.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\inesr> python.exe -m pip install --upgrade pip
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pip in c:\users\inesr\appdata\roaming\python\python313\site-packages (25.1.1)
Collecting pip
  Downloading pip-26.1.1-py3-none-any.whl.metadata (4.6 kB)
Downloading pip-26.1.1-py3-none-any.whl (1.8 MB)
  1.8/1.8 MB 18.4 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.1.1
    Uninstalling pip-25.1.1:
      Successfully uninstalled pip-25.1.1
  Successfully installed pip-26.1.1
PS C:\Users\inesr>
```

Figura 2: Instalación de OpenCV.

He accedido al repositorio oficial de código de OpenCV en GitHub (*opencv/data/haarcascades*) para descargar de forma local los clasificadores necesarios. En la Figura 3 se puede observar el contenido del repositorio. Para el ejercicio con reconocimiento de caras basta con descargar el fichero subrayado en naranja sobre rostros en posición frontal: *haarcascade_frontalface_alt.xml*. Para reconocer caras en videos hacen falta dos: *haarcascade_eye_tree_eyeglasses.xml* y *haarcascade_frontalface_default.xml*.

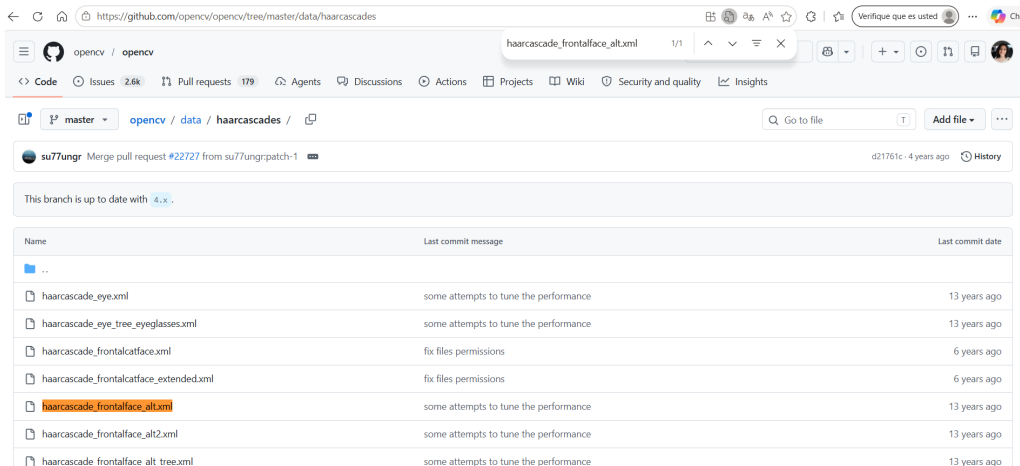


Figura 3: Repositorio OpenCV

3.1. Ejemplo de caras

El código fuente proporcionado por el profesor se encuentra en el Anexo 5.1. El material necesario (imagen y fichero *.xml*) están en la siguiente dirección del repositorio: <https://github.com/ruiz314/PDIH/blob/main/P6/ejemplos/caras/ejemplo-fotos-1.py>.

La imagen inicial es la que se muestra en la Figura 4. Muestra dos rostros de dos hombres en un primer plano. Al fondo de la imagen hay un cuadro con un rostro de otro hombre. La imagen está en blanco y negro.

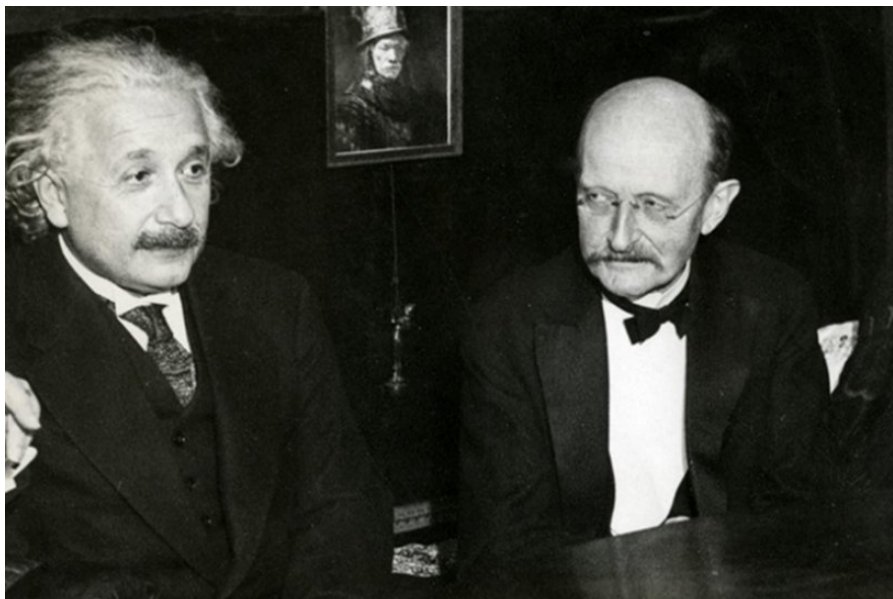


Figura 4: Imagen inicial

Para ejecutar este ejemplo en el que detecta las caras de una imagen basta con escribir *python* y el nombre del fichero:

```
python ejemplo-fotos-1.py
```

En la Figura 5 se puede observar el resultado de haber ejecutado el ejemplo proporcionado por el profesor. Solo se detecta una cara, la de la derecha (Max Planck). En un recuadro amarillo que he dibujado en mi terminal se puede ver el contenido de la carpeta, y subrayado por una línea amarilla viene el comando para ejecutar dicho programa.

En la Figura 6 se pueden ver dos recuadros rojos, es decir, el programa ha detectado dos caras en la imagen. La cara del cuadro no ha sido reconocida por el programa. Si reducimos el parámetro *scaleFactor* y le damos

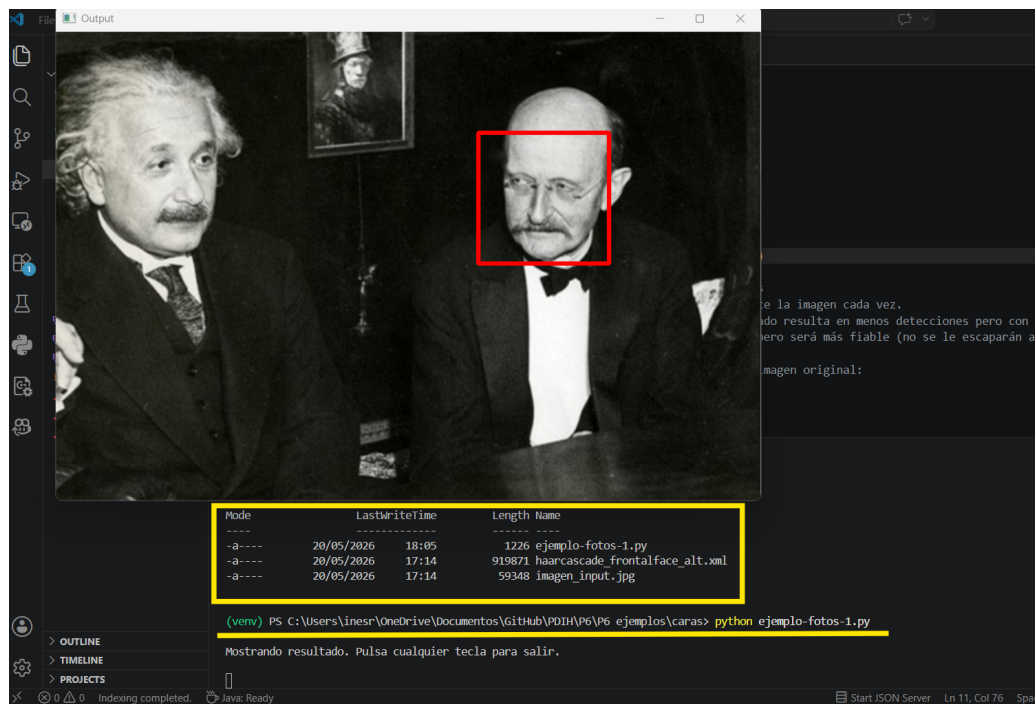


Figura 5: Ejecución de ejemplo

valor más cercano a 1,0 (p.ej. 1,05 en mi caso) veremos que sí identifica esa otra cara del cuadro. Subrayado en amarillo se encuentra el parámetro `scaleFactor`, para ver su valor en esa ejecución.

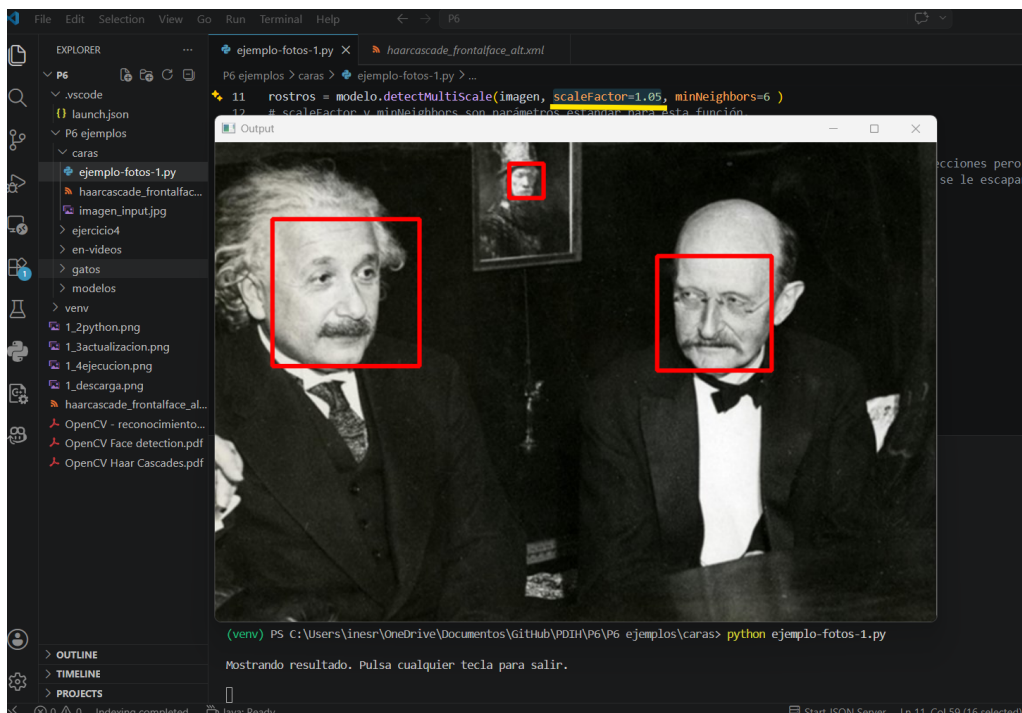


Figura 6: Ejecución de ejemplo con `scaleFactor = 1.05`

4. Ejercicios propuestos

4.1. Ejercicio 1

Para tener más imágenes con las que trabajar y probar el programa he buscado imágenes libres en la página *Pixabay*: <https://pixabay.com/es/photos/search/fotos%20de%20personas/>. A continuación presento las imágenes con las que voy a trabajar:

- Figura 7 - Dos personas en un parque. La persona de la izquierda lleva una gorra, tiene barba y está mirando de frente. La persona de la derecha tiene el pelo corto y casi está mirando de frente a la cámara.
- Figura 8 - Aparecen ocho personas, todas mirando a cámara y sonriendo.

Para esta tarea se ha empleado el clasificador frontal avanzado `haarcascade_frontalface_alt.xml`. También importo la librería `sys` para poder incluir la imagen con la que se trabaja como parámetro en la orden de ejecución. Esto facilita probar con varias imágenes sin tener que tocar el fichero de código fuente. Además, uso la librería `os` para guardar las imágenes resultantes (con caras recuadradas) en la carpeta de trabajo.

```
1 import cv2
2 import sys
3 import os
```

Listing 2: Librerías para el Ejercicio 1

Como la imagen con la que se va a trabajar se pasa por parámetro hay que comprobar que se haya pasado, que la ruta exista y que no esté corrupta:

```
1 # Validación del número de argumentos en la línea de comandos
2 if len(sys.argv) < 2:
3     print("[ERROR] Uso correcto: python ejercicio1.py <nombre_imagen.jpg>")
4     sys.exit(1)
5
6 # Captura del parámetro del archivo de imagen
7 nombre_imagen = sys.argv[1]
8
9 #Cargamos nuestro modelo clasificador de Haar:
10 modelo = cv2.CascadeClassifier('haarcascade_frontalface_alt.xml')
```




Figura 7: Imagen 1

```

12 #Cargamos la imagen
13 img = cv2.imread(nombre_imagen)
14
15 # Control de excepciones ante rutas inexistentes o corruptas
16 if img is None:
17     print(f"[ERROR] No se pudo cargar la imagen '{nombre_imagen}'")
18     sys.exit(1)

```

Listing 3: Comprobación de parametro

A continuación, el programa sigue como el del ejemplo, haciendo el procesamiento de escala de grises, buscando rostros y dibujando los rectángulos sobre la imagen original. Una vez hecho eso, se guarda la imagen en disco separando el nombre de la imagen de la extensión:

```

1 # Separamos el nombre del archivo de su extensión (ej: 'foto' y '.jpg')
2 nombre_base, extension = os.path.splitext(nombre_imagen)
3
4 # Creamos el nuevo nombre para el archivo de salida
5 nombre_salida = f"resultado_{nombre_base}{extension}"
6
7 # Guardamos la imagen modificada en la carpeta de trabajo
8 cv2.imwrite(nombre_salida, img)
9 print(f"[INFO] Imagen resultante guardada con éxito como: {nombre_salida}")

```

Listing 4: Guardar imagen

El resultado usando `scaleFactor=1.35`, `minNeighbors=6` en la Figura 7 es la Figura 9. El motivo de usar `scaleFactor=1.35` es porque con el valor 1,5 no se reconoce ningún rostro. El rostro del hombre no lo llega a reconocer aunque se disminuya el valor de `scaleFactor`.

El resultado usando `scaleFactor=1.5`, `minNeighbors=6` en la Figura 8 es la Figura 10. Se reconocen 5 rostros, de 8 que hay en la imagen.

Si disminuimos el valor de `scaleFactor` a 1,35 el programa reconocerá todas las caras, como se muestra en la Figura 11.



Figura 8: Imagen 2

5. Anexo

5.1. Fichero *ejemplo-fotos-1.py*

Ejemplo proporcionado por el profesor. En el repositorio de *GitHub* está en la siguiente ruta: <https://github.com/ruiz314/PDIH/blob/main/P6/ejemplos/caras/ejemplo-fotos-1.py>.

```

1 # ejemplo-fotos-1.py
2 import cv2
3
4 #Cargamos nuestro modelo classificador de Haar:
5 modelo = cv2.CascadeClassifier('haarcascade_frontalface_alt.xml')
6
7 #Cargamos la imagen
8 img = cv2.imread('imagen_input.jpg')
9 imagen = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
10
11 #Buscamos los rostros:
12 rostros = modelo.detectMultiScale(imagen, scaleFactor=1.5, minNeighbors=6 )
13 # scaleFactor y minNeighbors son parámetros estándar para esta función.
14 # El primero es el factor de escala: la función intentará encontrar rostros
15 # escalando la imagen varias veces, y este factor indica en cuánto se reduce la
16 # imagen cada vez.
17 # El segundo parámetro indica la calidad de las detecciones: un valor elevado resulta
18 # en menos detecciones pero con más fiabilidad.
19 # Si hacemos scaleFactor más cercano a 1.0 tardará un poco más de tiempo, pero será m
20 # ás fiable (no se le escaparán algunas caras)
21
22 #Ahora recorremos el array 'rostros' y dibujamos los rectángulos sobre la imagen
23 # original:
24 for (x,y,ancho, alto) in rostros:
25     cv2.rectangle(img, (x,y), (x+ancho, y+alto), (0,0,255) , 3)
26
27 #Abrimos una ventana con el resultado:
28 cv2.imshow('Output', img)
29 print("\nMostrando resultado. Pulsa cualquier tecla para salir.\n")
30 cv2.waitKey(0)

```



Figura 9: Resultado de *imagen1* con `scaleFactor=1.35`, `minNeighbors=6`

```
27 cv2.destroyAllWindows()
```

Listing 5: Código completo del ejemplo1

5.2. Fichero *ejercicio1.py*

```
1 import cv2
2 import sys
3 import os
4
5 # Validación del número de argumentos en la línea de comandos
6 if len(sys.argv) < 2:
7     print("[ERROR] Uso correcto: python ejercicio1.py <nombre_imagen.jpg>")
8     sys.exit(1)
9
10 # Captura del parámetro del archivo de imagen
11 nombre_imagen = sys.argv[1]
12
13 #Cargamos nuestro modelo classificador de Haar:
14 modelo = cv2.CascadeClassifier('haarcascade_frontalface_alt.xml')
15
16 #Cargamos la imagen
17 img = cv2.imread(nombre_imagen)
18
19 # Control de excepciones ante rutas inexistentes o corruptas
20 if img is None:
21     print(f"[ERROR] No se pudo cargar la imagen '{nombre_imagen}'")
22     sys.exit(1)
23
24 # Procesamiento en escala de grises
25 imagen = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
26
27 #Buscamos los rostros:
28 rostros = modelo.detectMultiScale(imagen, scaleFactor=1.35, minNeighbors=6 )
29
```



Figura 10: Resultado de *imagen2* con `scaleFactor=1.5`, `minNeighbors=6`

```

30 #Ahora recorremos el array 'rostros' y dibujamos los rectángulos sobre la imagen
    original:
31 for (x,y,ancho, alto) in rostros:
32     cv2.rectangle(img, (x,y), (x+ancho, y+alto), (0,0,255) , 3)
33
34 #Imprimir el número de rostros detectados
35 print(f"Número de rostros detectados con éxito: {len(rostros)}")
36
37 #Guardar la imagen en disco
38 # Separamos el nombre del archivo de su extensión (ej: 'foto' y '.jpg')
39 nombre_base, extension = os.path.splitext(nombre_imagen)
40 # Creamos el nuevo nombre para el archivo de salida
41 nombre_salida = f"resultado_{nombre_base}{extension}"
42
43 # Guardamos la imagen modificada en la carpeta de trabajo
44 cv2.imwrite(nombre_salida, img)
45 print(f"[INFO] Imagen resultante guardada con éxito como: {nombre_salida}")
46
47 #Abrimos una ventana con el resultado:
48 cv2.imshow(f'Output - {nombre_imagen}', img)
49 print("\nMostrando resultado del ejercicio 1. Pulsa cualquier tecla para salir.\n")
50 cv2.waitKey(0)
51 cv2.destroyAllWindows()

```

Listing 6: Código completo del ejercicio1

5.3. Fichero *ejemplo2.py*

Ejemplo proporcionado por el profesor. En el repositorio de *GitHub* está en la siguiente ruta: <https://github.com/ruiz314/PDIH/blob/main/P6/ejemplos/en-videos/ejemplo2.py>.

```

1 import cv2
2
3 # Cargamos el modelo clasificador para la cara
4 modeloCara = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
5
6 # Abrimos la cámara o cargamos un vídeo:
7 #camara = cv2.VideoCapture(-1)

```

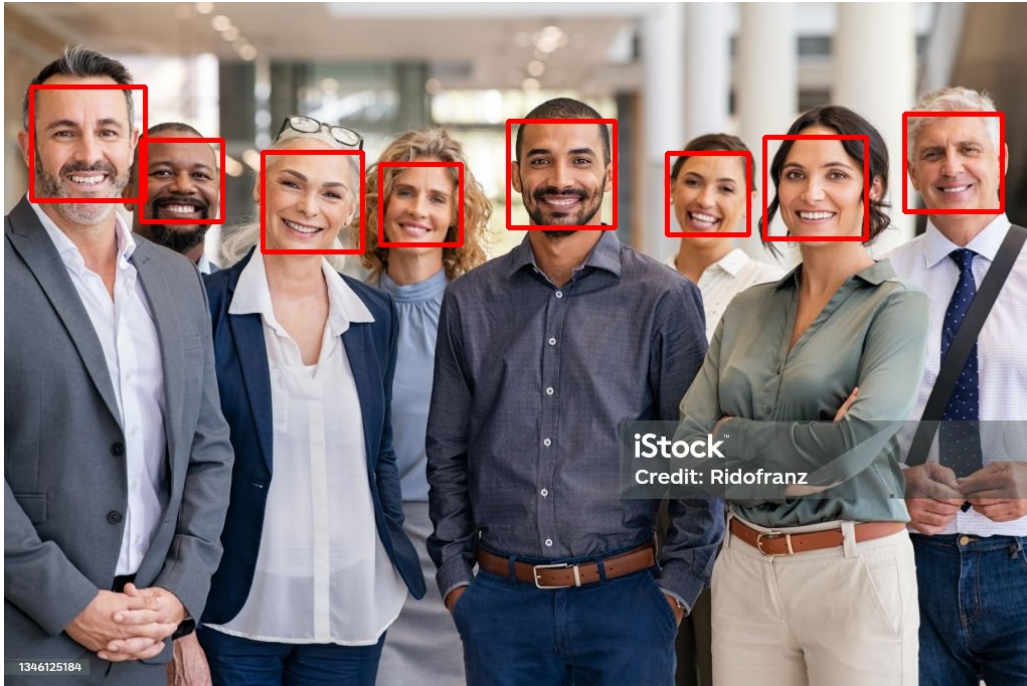



Figura 11: Resultado de *imagen2* con `scaleFactor=1.35`, `minNeighbors=6`

```

8  camara = cv2.VideoCapture('v2.mp4')
9
10 while camara.isOpened():
11     # Cargamos en img lo capturado por la cámara
12     _, img = camara.read()
13
14     color = cv2.cvtColor(img, cv2.COLOR_BGR2BGRA)
15
16     # Detectamos la cara
17     faces = modeloCara.detectMultiScale(color, 1.1, 4)
18
19     # Pintamos el rectángulo sobre la cara
20     for (x, y, w, h) in faces:
21         cv2.rectangle(img, (x,y), (x+w, y+h), (255, 0, 0), 3)
22
23     # Lo mostramos en pantalla mientras no pulsemos la tecla 'esc'
24     cv2.imshow('img', img)
25     key = cv2.waitKey(1)
26     if key == 27:
27         break
28
29 camara.release()
30 cv2.destroyAllWindows()

```

Listing 7: Código completo del ejemplo2

5.4. Fichero *ejemplo3.py*

Ejemplo proporcionado por el profesor. En el repositorio de *GitHub* está en la siguiente ruta: <https://github.com/ruiz314/PDIH/blob/main/P6/ejemplos/en-videos/ejemplo3.py>.

```

1  import cv2
2
3  # Cargamos clasificadores para cara y ojos
4  modeloCara = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
5  modeloOjo = cv2.CascadeClassifier('haarcascade_eye_tree_eyeglasses.xml')
6

```

```

7 # abrimos el flujo de vídeo desde la cámara o desde un vídeo
8 #camara = cv2.VideoCapture(-1)
9 camara = cv2.VideoCapture('v3.mp4')
10
11 while camara.isOpened():
12     _, img = camara.read()
13     color = cv2.cvtColor(img, cv2.COLOR_BGR2BGRA)
14
15     faces = modeloCara.detectMultiScale(color, 1.3, 4)
16     for (x, y , w ,h) in faces:
17         # Dibujamos rectángulo en las caras detectadas
18         cv2.rectangle(img, (x,y), (x + w, y + h), (255, 0 , 0), 3)
19
20     eyes = modeloOjo.detectMultiScale(color, 1.3, 4)
21     for (x, y , w ,h) in eyes:
22         # Dibujamos rectángulo en los ojos
23         cv2.rectangle(img, (x,y), (x + w, y + h), (0, 255 , 0), 3)
24
25     cv2.imshow('img', img)
26     key = cv2.waitKey(1)
27     if key == 27:
28         break
29
30 camara.release()
31 cv2.destroyAllWindows()

```

Listing 8: Código completo del ejemplo3